

Compararea experimentală a unor algoritmi de sortare implementați în Python

Adrian-Sorin Onofrei

Facultatea de Matematică și Informatică

Universitatea de Vest din Timișoara

Informatică în limba română, grupa 4

`adrian.onofrei06@e-uvv.ro`

Rezumat

Sortarea este una dintre problemele fundamentale din informatică și apare frecvent în aplicații care prelucrează date, caută informații sau organizează colecții de elemente. Deși există foarte mulți algoritmi de sortare, performanța lor diferă semnificativ în funcție de dimensiunea datelor și de modul în care acestea sunt distribuite. Din acest motiv, alegerea unui algoritm potrivit nu este doar o problemă teoretică, ci și una practică.

În această lucrare sunt analizate șase metode de sortare implementate în limbajul Python: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort și Counting Sort. Compararea a fost făcută pe mai multe tipuri de liste: liste aleatoare, deja sortate, sortate invers, aproape sortate și liste cu multe valori repetate. Pentru fiecare situație s-a măsurat timpul mediu de execuție.

Rezultatele confirmă diferențele dintre complexitățile teoretice și comportamentul practic al algoritmilor. Algoritmii de complexitate $O(n^2)$ devin rapid ineficienți pentru liste mai mari, în timp ce algoritmii mai eficienți, cum sunt Merge Sort, Quick Sort și Counting Sort, oferă timpi mult mai buni. În plus, se observă că anumite tipuri de date favorizează unii algoritmi, de exemplu Insertion Sort pe liste aproape sortate sau deja ordonate.

1 Introducere

1.1 Motivație

Problema sortării este importantă deoarece apare în multe domenii ale informaticii: baze de date, procesare de fișiere, căutare, analiză de date și optimizarea altor algoritmi. Deși la prima vedere sortarea pare o operație simplă, în practică există diferențe mari între metodele folosite.

Acest proiect a fost realizat pentru a compara în mod concret mai mulți algoritmi de sortare și pentru a observa:

- cum se comportă aceștia pe date de dimensiuni diferite;
- cum influențează structura datelor timpul de execuție;
- cât de bine se confirmă în practică analiza teoretică a complexității.

1.2 De ce este relevantă problema

În teorie, două metode pot rezolva aceeași problemă corect, dar în practică una poate fi mult mai rapidă decât alta. Pentru un student aflat la început, această diferență este importantă deoarece arată de ce complexitatea algoritmilor nu este doar un concept abstract, ci are efect direct asupra performanței unui program.

1.3 Soluții existente și limitele lor

Există algoritmi simpli, ușor de înțeles și implementat, cum sunt Bubble Sort, Insertion Sort și Selection Sort. Aceștia sunt utili didactic, dar devin lenți pentru intrări mari. Există și algoritmi mai eficienți, cum sunt Merge Sort și Quick Sort, iar în anumite condiții Counting Sort poate fi și mai rapid. Totuși, niciun algoritm nu este optim în orice situație, iar performanța depinde de natura datelor de intrare.

1.4 Contribuția lucrării

Contribuția acestei lucrări constă în:

- implementarea în Python a șase algoritmi de sortare;
- definirea mai multor tipuri de date de test;
- măsurarea experimentală a timpilor de execuție;
- analiza comparativă a rezultatelor.

1.5 Organizarea lucrării

Lucrarea este structurată astfel: Secțiunea 2 prezintă formal problema și proprietățile urmărite, Secțiunea 3 descrie implementarea și modul de rulare, Secțiunea 4 prezintă metodologia experimentală și rezultatele, Secțiunea 5 discută relația cu alte abordări, iar Secțiunea 6 conține concluziile și direcțiile de dezvoltare ulterioară.

2 Descriere formală a problemei și a soluției

2.1 Definirea problemei

Fie o listă

$$A = [a_1, a_2, \dots, a_n]$$

formată din numere întregi. Problema sortării constă în construirea unei permutări

$$A' = [b_1, b_2, \dots, b_n]$$

astfel încât:

1. elementele din A' să fie exact aceleași ca în A ;
2. să fie respectată relația de ordine:

$$b_1 \leq b_2 \leq \dots \leq b_n.$$

2.2 Date de intrare și date de ieșire

Intrare: o listă de numere întregi.

Ieșire: aceeași listă, ordonată crescător.

2.3 Proprietăți urmărite

Pentru a considera corectă o metodă de sortare, trebuie să fie îndeplinite două proprietăți:

- **corectitudine:** rezultatul final este sortat crescător;
- **conservarea elementelor:** lista rezultată conține exact aceleași valori ca lista inițială.

2.4 Algoritmii analizați

În acest proiect au fost studiați următorii algoritmi:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort
- Counting Sort

2.5 Complexități teoretice

Tabelul 1 prezintă complexitățile teoretice ale algoritmilor utilizați.

Tabela 1: Complexități teoretice			
Algoritm	Caz favorabil	Caz mediu	Caz nefavorabil
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$

Aici, n reprezintă numărul de elemente, iar k reprezintă intervalul valorilor posibile pentru Counting Sort.

3 Modelarea problemei și implementarea soluției

3.1 Limbajul folosit

Toți algoritmi au fost implementați în limbajul Python, deoarece oferă o sintaxă clară și este potrivit pentru experimente de laborator și proiecte introductive.

3.2 Structura programului

Programul conține:

- implementarea fiecărui algoritm de sortare;
- funcții pentru generarea seturilor de date;
- o funcție pentru măsurarea timpului de execuție;
- o funcție principală care rulează toate testele și scrie rezultatele într-un fișier text.

3.3 Tipuri de date generate

Au fost utilizate următoarele categorii de liste:

- **random** – liste cu valori generate aleator;
- **sorted** – liste deja sortate crescător;
- **reverse** – liste ordonate descrescător;
- **nearly** – liste aproape sortate, cu puține interschimbări;
- **flat** – liste cu multe valori repetitive.

3.4 Măsurarea performanței

Pentru măsurarea timpului de execuție s-a folosit funcția `time.perf_counter()`, deoarece oferă o precizie bună pentru măsurători scurte. Pentru fiecare combinație algoritm-tip de date-dimensiune s-au făcut 5 rulări, iar rezultatul folosit în analiză a fost media lor aritmetică.

3.5 Fragment reprezentativ de cod

Mai jos este prezentat un fragment relevant din implementare:

Listing 1: Măsurarea timpului de execuție

```
def measure(func, data):  
    t0 = time.perf_counter()  
    func(data)  
    return time.perf_counter() - t0
```

3.6 Instrucțiuni de rulare

Programul poate fi rulat direct din terminal cu:

```
python nume_fisier.py
```

La final, rezultatele sunt salvate în fișierul `rezultate.txt`.

4 Studiu experimental

4.1 Metodologie

Experimentul a fost conceput pentru a compara comportamentul practic al algoritmilor de sortare în condiții cât mai variate. Au fost testate dimensiunile:

10, 50, 100, 1000, 5000, 10000.

Nu s-au folosit liste foarte mari (de ordinul sutelor de mii sau milioane de elemente), deoarece algoritmii de complexitate $O(n^2)$ devin impracticabili în astfel de condiții, atât din cauza timpului de execuție, cât și din cauza limitărilor hardware.

4.2 Rezultate reprezentative

Pentru a păstra documentul clar, în continuare sunt prezentate două tabele sintetice, urmate de interpretarea lor.

Tabela 2: Timp de execuție pentru date `random`, $n = 10000$

Algoritm	Timp mediu (sec)
Bubble Sort	3.525053
Insertion Sort	1.595017
Selection Sort	1.532276
Merge Sort	0.016754
Quick Sort	0.013402
Counting Sort	0.001970

Tabela 3: Timp de execuție pentru toate tipurile de date, $n = 10000$

Algoritm	random	sorted	reverse	nearly	flat
Bubble	3.525053	2.080148	4.513767	2.246732	3.221649
Insertion	1.595017	0.000789	3.175553	0.198003	1.283585
Selection	1.532276	1.461771	1.575399	1.467162	1.596127
Merge	0.016754	0.010204	0.010942	0.014484	0.014685
Quick	0.013402	0.008653	0.008974	0.009451	0.001559
Counting	0.001970	0.001859	0.001883	0.001921	0.000570

4.3 Analiza rezultatelor

Pe baza rezultatelor obținute, se observă următoarele aspecte importante:

1. **Bubble Sort** este cel mai lent algoritm în aproape toate testele. Acest lucru era de așteptat, deoarece face foarte multe comparații și interschimbări.
2. **Insertion Sort** are un comportament foarte bun pe liste sortate sau aproape sortate. De exemplu, pentru liste sortate cu $n = 10000$, timpul este foarte mic în comparație cu celelalte cazuri.
3. **Selection Sort** are o performanță relativ constantă, dar slabă. Nu profită de faptul că datele sunt deja ordonate.
4. **Merge Sort** oferă rezultate bune și stabile, indiferent de tipul de intrare. Acest lucru corespunde complexității sale de ordin $O(n \log n)$ în toate cazurile.
5. **Quick Sort** este foarte rapid în practică și, în testele realizate, a avut performanțe apropiate sau mai bune decât Merge Sort.
6. **Counting Sort** este cel mai rapid algoritm în aproape toate situațiile testate. Totuși, acest rezultat trebuie interpretat ținând cont de faptul că algoritmul este potrivit doar când valorile sunt întregi și intervalul lor nu este foarte mare.

4.4 Observații speciale

Un rezultat interesant apare în cazul listelor de tip **flat**, unde există multe valori repetate. În această situație, Quick Sort și Counting Sort obțin timpi foarte buni. De asemenea, Insertion Sort devine foarte eficient când lista este deja aproape ordonată, ceea ce confirmă comportamentul teoretic al algoritmului.

5 Related work / Lucrări conexe

Problema sortării este intens studiată în literatura de specialitate și apare în aproape toate cursurile introductive de algoritmi. Algoritmii clasici, precum Bubble Sort, Insertion Sort și Selection Sort, sunt folosiți în principal pentru înțelegerea tehnicilor de bază. Algoritmii mai eficienți, precum Merge Sort și Quick Sort, sunt considerați standarde importante în studiul proiectării algoritmilor.

Din punct de vedere al comparației dintre metode, se pot observa următoarele avantaje și dezavantaje:

Tabela 4: Avantaje și dezavantaje ale algoritmilor analizați

Algoritm	Avantaje	Dezavantaje
Bubble Sort	Foarte ușor de înțeles și implementat	Foarte lent pentru liste mari
Insertion Sort	Bun pentru liste mici sau aproape sortate	Slab pentru liste mari, în special în caz nefavorabil
Selection Sort	Implementare simplă, număr redus de interschimbări	Timp mare, nu profită de ordonarea existentă
Merge Sort	Stabil, eficient și predictibil	Folosește memorie suplimentară
Quick Sort	Foarte rapid în practică	Poate avea caz nefavorabil $O(n^2)$
Counting Sort	Extrem de rapid în condiții potrivite	Nu este general; depinde de intervalul valorilor

Astfel, lucrarea de față se încadrează într-o direcție clasică de analiză experimentală a algoritmilor, punând accent pe compararea practică a performanței și nu doar pe prezentarea teoretică.

6 Concluzii și direcții viitoare

6.1 Concluzii

În urma experimentelor realizate, se pot formula următoarele concluzii:

- algoritmii de complexitate $O(n^2)$ sunt potriviți mai ales pentru învățare și pentru intrări mici;
- pentru liste mai mari, Merge Sort și Quick Sort sunt opțiuni mult mai bune;
- Counting Sort este extrem de eficient atunci când datele respectă condițiile necesare;
- structura datelor de intrare influențează puternic performanța anumitor algoritmi;
- analiza practică confirmă, în mare parte, complexitățile teoretice studiate la curs.

6.2 Direcții viitoare

Acest proiect poate fi extins în mai multe moduri:

- testarea pe seturi de date mai mari;
- compararea cu funcția `sorted()` din Python;
- reprezentarea grafică a rezultatelor;
- analiza consumului de memorie, nu doar a timpului;
- adăugarea altor algoritmi, cum ar fi Heap Sort sau Radix Sort.

Bibliografie

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press, ediția a 3-a, 2009.
- [2] Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1998.
- [3] Robert Sedgewick, Kevin Wayne, *Algorithms*, Addison-Wesley, ediția a 4-a, 2011.